

프리지아 랩

[Home](#)[Location](#)[Tags](#)[Media](#)[Guestbook](#)[Admin](#)[New Pc](#)

교육 & 세미나

Container & DevOps - DAY 1

by 강철 벚꽃 2026. 6. 23.

day1.zip
0.02MB05_docker-network.azcli
0.00MB

DAY 1

- 로컬 컨테이너 개발 환경 준비
- 컨테이너 기술과 도커 개요
- 도커 이미지 다루기
- Dockerfile과 이미지 빌드
- 도커 컨테이너 다루기
- 도커 컴포즈

더 프리지아 랩

Azure & Windows

Windows 10

Windows Server

Azure

PowerShell

Localization Issue

Programming

Visual Studio

Mixed Reality

Windows Phone

Exchange & Outlook

Trouble Shooting

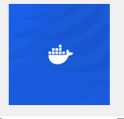
System Administration

Development

Outlook Life

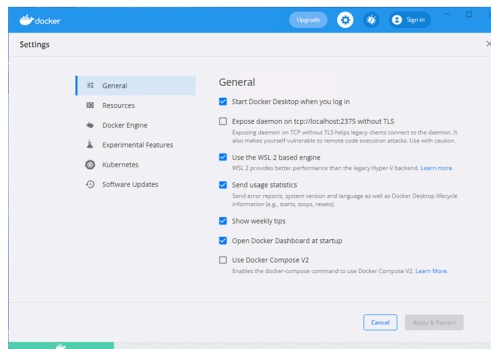
My Favorite Tools

컨테이너 개발 환경



컨테이너 개발 환경 | Docker Desktop

1. 운영 체제 준비
2. Docker Desktop 다운로드
3. Docker Desktop 설치
4. Docker Desktop 설정
5. Docker 동작 확인
6. VS Code Docker 확장 설치



6

컨테이너 기술과 도커 개요



컨테이너 기술 개요

■컨테이너 개념

- ✓사전적의미 - '물체를 격리하는 공간'
- ✓컴퓨팅 용어 - '모듈화 되고 격리된 컴퓨팅 환경'
- ✓가상화 용어 - '앱과 앱을 구동하는 환경을 격리한 공간'

■컨테이너형 가상화 기술

- ✓가상화 SW 없이 OS의 리소스를 격리해 앱을 실행하는 가상 운영 체제 공간을 만드는 기술

■컨테이너형 가상화 기술(OS 수준 가상화 기술)

- ✓컨테이너형 가상화 기술의 등장 - 1982, chroot
- ✓본격적인 사용 - 2008, LXC(Linux Container)
- ✓LXC의 문제 - 구동환경(네트워크, 스토리지, 보안 등) 차이에 따른 잦은 실행 문제

8

TechSmith

교육 & 세미나

기술 세미나

메타넷

가톨릭 대학교

남부여성발전센터

미쓰비시전기오토메이션

미림 마이:스터고

세완C교육

부산정보산업진흥원

Microsoft

NHN Cloud

Books

강철벼룩의 서재

출간 소식

번역 - 활자에 날을 세운다

LifeStyle INNOVATOR

경제지능 업그레이드

그들의 공간

가족 소식

경험 공유

Trend Or Future

공지사항

한국마이크로소프트

MAICPP Readines...

[LINC3.0사업단]알쓸챗
(Chat)잡 교육...

클라우드 및 인공지능 기술
책 저자들과 함께...

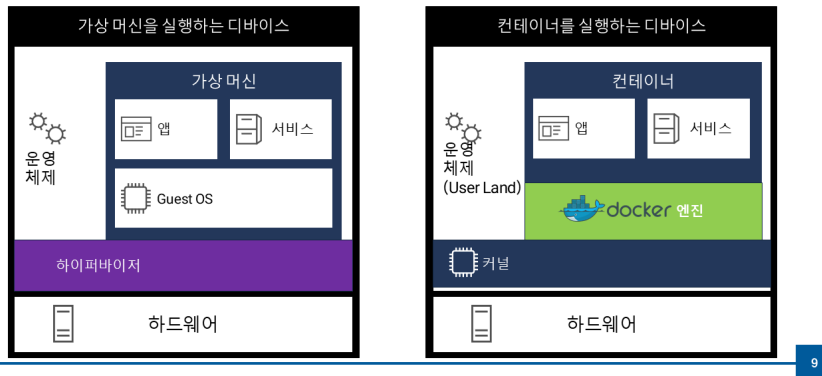
한남 대학교 특강

제 11회 공가 SW 개발자 대
회 멘토로 위촉...

최근글 인기글

Container & DevOps -
DAY 1

가상화 vs. 컨테이너



도커 서비스 개요

도커(Docker)란?

- ✓ 컨테이너형 가상화 기술을 구현하기 위한 상주 애플리케이션과 이 애플리케이션을 조작하기 위한 명령 줄 도구로 구성된 제품
- ✓ 초기에는 LXC를 커널의 가상화 기능 액세스 인터페이스로 사용
- ✓ 0.9x 부터 자체 개발한 libcontainer로 LXC 대체

도커의 등장

- ✓ 프랑스의 dotCloud의 내부 프로젝트로 시작(솔로몬 하익스)
- ✓ Docker, Inc. 설립 - Y 컴비네이터 2010 여름 스타트업 인큐베이터 그룹 ([Solomon Hykes](#)/Sebastien Pahl)
- ✓ 2013년 3월 오픈소스로 출시

도커 구성 요소와 도구

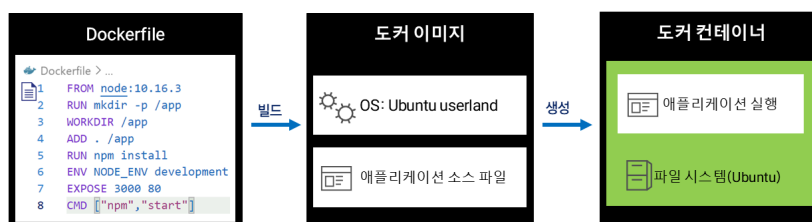
구성 요소

- ✓ Docker Engine (run time): dockerd ← docker engine api
- ✓ Docker Client (CLI): docker
- ✓ Docker Objects: Docker container, Docker Image, Docker service
- ✓ Docker Registry

도구

- ✓ Docker Compose: docker-compose, docker-compose.yml
- ✓ Docker Swarm: docker swarm CLI, docker node CLI

Dockerfile, Docker Image, Docker Container의 관계



2026.06.23



웹 프로그래밍 개요 - DAY 1

2026.06.22

"The content f...

2026.01.01



Azure VM의 Ubuntu...

2025.11.09



로컬로

Azure...

2025.05.06



최근댓글

강철 베희님 공감 꼭 누르...

물론이죠. 나가 연락할게요.

행복한 오늘 되세요~공감...

답변 정말 감사합니다 다음...

태그

VS2019, 윈도우 폰7,

PowerShell,

visual studio code,

회의실, azure,

azure powershell,

localization,

Windows 8, 스타트업,

Azure AD, ca,

코드검사분석, RAS,

아웃룩 2007, 저널링,

지앤선, Nano Server,

header, 윈도우 폰, blob,

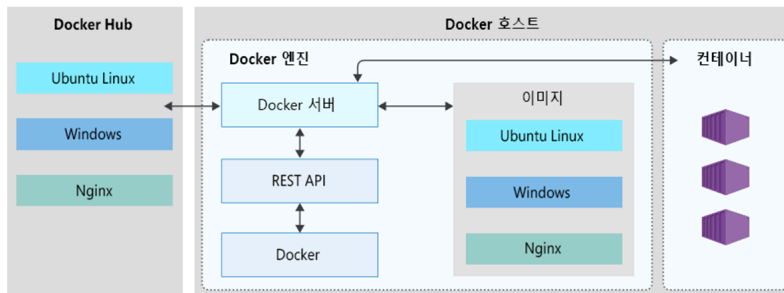
정보문화사, vscode,

오프라인 주소록, uwp,

실행모델, lam,

windows 10,

Docker 환경 아키텍처



13

Developer Preview,
Windows Phone

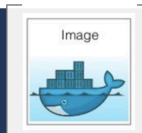
전체 방문자

1,096,471

Today : 7

Yesterday : 81

Docker Image 다루기



Docker 이미지 관리 주요 명령 1

▪ docker search

- ✓ 도커 허브에 등록된 리포지토리 검색
- ✓ 공식 리포지토리 네임스페이스는 생략가능
- ✓ docker search [options] 검색어

```
PS D:\DockerClass> docker search --limit 5 mysql
```

NAME	DESCRIPTION	STARS	OFFICIAL	AUTOMATED
mysql	MySQL is a widely used, open-source relation...	8817	[OK]	
mysql/mysql-server	Optimized MySQL Server Docker images. Create...	654		[OK]
mysql/mysql-cluster	Experimental MySQL Cluster Docker images. Cr...	56		
bitnami/mysql	Bitnami MySQL Docker Image	35		[OK]
circleci/mysql	MySQL is a widely used, open-source relation...	15		

15

Docker 이미지 관리 주요 명령 2

▪ docker image pull

- ✓ docker pull
- ✓ 도커 레지스트리에서 이미지 내려받기
- ✓ docker image pull <옵션> <리포지토리[:태그]>

▪ docker image ls

- ✓ docker images
- ✓ 호스트에 저장된 이미지 목록
- ✓ docker image ls <옵션> <리포지토리[:태그]>

```
PS D:\DockerClass> docker image pull mongo:latest
latest: Pulling from library/mongo
7ddbc47eeb70: Pull complete
c1bbdc448b72: Pull complete
8c3b70e39044: Pull complete
45d437916d57: Pull complete
e119fb0e0a55: Pull complete
91f0b9bae1ea: Pull complete
53e7c2967f11: Pull complete
69a945568374: Pull complete
93333bc225a7: Pull complete
b9c10bd6c9bd: Pull complete
7f4e3538e99c: Pull complete
1164b51d180a: Pull complete
a715a7d71f27: Pull complete
Digest: sha256:2704b1f2ad53c0c5fb029fc112f99b5e9acdc
Status: Downloaded newer image for mongo:latest
docker.io/library/mongo:latest
```

16

Docker 이미지 관리 주요 명령 3

▪ docker image tag

- ✓도커 이미지의 버전은 이미지 ID
- ✓이미지 ID에 붙이는 별명 (쉬운 식별)
- ✓태그 하나 당 이미지 하나
- ✓docker image tag <원본 이미지[:태그]> <새 이미지[:태그]>

```
PS D:\DockerClass> docker image tag guestbook_node_dockerapp:latest steelflea/simple_guestbook_express:lates
PS D:\DockerClass> docker image ls
```

REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
guestbook_node_dockerapp	latest	0133636591f2	9 hours ago	905MB
steelflea/simple_guestbook_express	latest	0133636591f2	9 hours ago	905MB

17

실습 - 도커 명령으로 이미지 다루기

Dockerfile과 이미지 빌드



Dockerfile 문법 1

▪ 전용 도메인 언어(명령, 인스트럭션)로 이미지 구성 정의

▪ FROM <이미지명>[:<태그>]

▪ RUN <도커 컨테이너 안에서 실행할 명령>

▪ COPY <원본 파일/디렉터리> <대상 디렉터리>

▪ CMD

- ✓문법 1 - CMD ["명령1" "인자1" "인자2" ...]
- ✓문법 2 - CMD <명령> <인자1> <인자2>

```
Dockerfile > ...
1 FROM node:8.9.3-alpine
2 RUN mkdir -p /usr/src/app
3 COPY ./app/* /usr/src/app/
4 WORKDIR /usr/src/app
5 RUN npm install
6 CMD node /usr/src/app/index.js
```

20

Dockerfile 문법 1

- ADD
 - ✓ 문법 1 - ADD <원본> <대상>
 - ✓ 문법 2 (공백이 있는 경우) - ADD ["<원본>", "<대상>"]
- ENV <환경변수명> <값>
- EXPOSE <포트1> <포트2>
- WORKDIR <작업 디렉터리 경로>
- LABEL <이미지를 만든 사람에 대한 정보>

```
Dockerfile > ...
1 FROM node:10.16.3
2 RUN mkdir -p /app
3 WORKDIR /app
4 ADD . /app
5 RUN npm install
6 ENV NODE_ENV development
7 EXPOSE 3000 80
8 CMD ["npm", "start"]
```

21

Docker 이미지 빌드

- docker image build <Dockerfile>
 - ✓ 태그 생략 시 → latest
 - ✓ [-f <다른 파일명>] → Dockerfile이 아닌 경우
 - ✓ [-pull = true] → 베이스 이미지를 매번 다시 받음
 - ✓ 사용자 네임스페이스 추가 → 이미지 명 충돌 방지
- Examples/guestbook4node

```
PS D:\DockerClass\GuestBook4ExpressJS> docker build . -t guestbook
Sending build context to Docker daemon  2.154MB
Step 1/8 : FROM node:10.16.3
--> a68faf70e589
Step 2/8 : RUN mkdir -p /app
--> Using cache
--> 4fead7f58281
Step 3/8 : WORKDIR /app
--> Using cache
--> 0eb50f01b1c
Step 4/8 : ADD . /app
--> b8cd50ac6a2e
Step 5/8 : RUN npm install
--> Running in 227f207aa911
audited 141 packages in 1.575s
found 0 vulnerabilities
Removing intermediate container 227f207aa911
--> 18aed218feac
Step 6/8 : ENV NODE_ENV development
--> Running in 767669a157ah
PS D:\DockerClass\GuestBook4ExpressJS> docker image ls
```

REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
guestbook_node_dockerapp	latest	0133636591f2	3 hours ago	905MB
firstnodedockerapp	latest	39f10ce09d9c	3 hours ago	905MB
expressguestbook	latest	a81f1b4781c6	15 hours ago	906MB
aci-tutorial-app	latest	534fa9ad8208	18 hours ago	71MB
node	10.16.3	a68faf70e589	4 weeks ago	904MB
node	8.9.3-alpine	144aaf4b1367	23 months ago	67.7MB

22

실습 - 도커 이미지 빌드하기

Docker 컨테이너 다루기



Docker 컨테이너 수명 주기

■ 실행 중

- ✓ Dockerfile의 CMD/ENTRYPOINT 인스트럭션에 정의된 앱 실행 중 상태

■ 정지

- ✓ 사용자에 의한 명시적인 중단
- ✓ 정상적인 실행의 종료
- ✓ 컨테이너 종료 시점의 상태를 디스크에 저장(재실행)

■ 파기

- ✓ 디스크 공간 확보를 위한 명시적인 완전 삭제
- ✓ 동일한 이미지로 다시 실행한 경우 완전히 다른 컨테이너

25

Docker 컨테이너 관리 주요 명령 1

■ docker container run

- ✓ Docker 이미지에서 컨테이너 생성 및 실행
- ✓ docker container run <옵션> <이미지명> [명령] [명령인자...]
- ✓ docker container run <옵션> <이미지 ID> [명령] [명령인자...]
- ✓ Dockerfile 인스트럭션 재정의

```
PS D:\DockerClass> docker container run -it alpine:3.7
/ # exit
PS D:\DockerClass> docker container run -it alpine:3.7 uname -a
Linux bfc1a932075b 4.9.184-linuxkit #1 SMP Tue Jul 2 22:58:16 UTC 2019 x86_64 Linux
```

■ 컨테이너에 원하는 이름 붙이기

- ✓ 자동 이름 대신 알기 쉬운 이름으로 개발 생산성 향상
- ✓ docker container run --name <컨테이너명> <이미지명[:태그]>

```
PS D:\DockerClass> docker container run -d -p 8081:3000 --name guestbookapp guestbook_node_dockerapp:latest
7c6426ba822ec186fb4809ef8cf1c108950f395950dcaab5c9e2fe4375419084
```

26

Docker 컨테이너 관리 주요 명령 2

■ 포그라운드 실행(attached mode)

- ✓ docker container run <이미지명:태그명>
- ✓ 컨테이너 종료 - <Ctrl + C>

■ 포트 포워딩

- ✓ 컨테이너 밖의 요청을 컨테이너 안으로 전달
- ✓ -p HOST_PORT:CONTAINER_PORT
- ✓ 호스트 포트 생략 가능 - 자동 할당

■ 백그라운드 실행(detached mode)

- ✓ docker container run -d <이미지명:태그명>

✓ 표준 출력 □ 컨테이너 ID

```
PS D:\DockerClass> docker container run -d -p 8080:3000 firstnodeockerapp
9ab24226bf94b07e28f8a6021821a6b5484eebec8f8bc5e67edbb48fa24d2cfe3b3b7f36c4e73ea4
PS D:\DockerClass> docker container run -d -p 8080 firstnodeockerapp
c81e6021821a6b5484eebec8f8bc5e67edbb48fa24d2cfe3b3b7f36c4e73ea4
PS D:\DockerClass> docker container ls
CONTAINER ID        IMAGE               COMMAND                  CREATED        STATUS        PORTS
c81e6021821a        firstnodeockerapp  "docker-entrypoint.s..." 15 seconds ago Up 13 seconds 80/tcp, 3000/tcp, 0.0.0.0:32768->8088/tcp
9ab24226bf94        firstnodeockerapp  "docker-entrypoint.s..." 50 seconds ago Up 48 seconds 80/tcp, 0.0.0.0:8080->3000/tcp
954e3a6510ec        guestbook_node_dockerapp  "docker-entrypoint.s..." 4 hours ago   Up 4 hours   80/tcp, 0.0.0.0:8081->3000/tcp
PS D:\DockerClass> []
```

27

Docker 컨테이너 관리 주요 명령 3

■ docker container ls = docker ps

- ✓ 실행 중/종료된 컨테이너 목록 표시
- ✓ -q : 컨테이너 ID만 추출
- ✓ --filter "필터명=값"
- ✓ -a: 종료된 컨테이너 목록

```
PS D:\DockerClass> docker container ls -q
7c6426ba822e
9ab24226bf94
PS D:\DockerClass> docker container ls --filter "name=guestbookapp"
CONTAINER ID        IMAGE               COMMAND                  CREATED        STATUS        PORTS        NAMES
7c6426ba822e        guestbook_node_dockerapp:latest  "docker-entrypoint.s..." 12 minutes ago Up 12 minutes 80/tcp, 0.0.0.0:8081->3000/tcp  guestbookapp
PS D:\DockerClass> docker container ls --filter "ancestor=firstnodeockerapp"
CONTAINER ID        IMAGE               COMMAND                  CREATED        STATUS        PORTS        NAMES
9ab24226bf94        firstnodeockerapp  "docker-entrypoint.s..." 6 hours ago   Up 6 hours   80/tcp, 0.0.0.0:8080->3000/tcp  suspicious_base
PS D:\DockerClass> docker container ls -a
CONTAINER ID        IMAGE               COMMAND                  CREATED        STATUS        PORTS        NAMES
7c6426ba822e        guestbook_node_dockerapp:latest  "docker-entrypoint.s..." 14 minutes ago Up 14 minutes 80/tcp, 0.0.0.0:8081->3000/tcp  ga
bfc1a932075b        alpine:3.7         "uname -a"              29 minutes ago Exited (0) 28 minutes ago
```

28

Docker 컨테이너 관리 주요 명령 4

■ 실행 중인 컨테이너 종료

- ✓ docker container stop <컨테이너ID/컨테이너명>

■ 컨테이너 재시작

- ✓ docker container restart <컨테이너ID/컨테이너명>

■ 컨테이너 파기

- ✓ docker container rm [-f (실행 중인 컨테이너 강제 삭제)] <컨테이너ID/컨테이너명>

```
PS D:\DockerClass> docker container rm bfc1a932075b
PS D:\DockerClass> docker container rm zen_easley
```

```
PS D:\DockerClass> docker container stop guestbookapp
guestbookapp
PS D:\DockerClass> docker container ls
CONTAINER ID        IMAGE               COMMAND
PS D:\DockerClass> docker container restart 9ab24226bf94
9ab24226bf94
```

29

Docker 컨테이너 관리 주요 명령 5

■ 표준 출력 연결하기

- ✓ 실행 중인 특정 도커 컨테이너의 표준 출력 내용 확인
- ✓ docker container logs -f(표준 출력 내용을 계속 볼 경우) <컨테이너ID/컨테이너명>

■ 실행 중인 컨테이너에서 명령 실행

- ✓ 컨테이너에 SSH 로그인한 것처럼 컨테이너 조작
- ✓ docker container exec <옵션> <컨테이너ID/컨테이너명> <컨테이너에서 실행할 명령>
- ✓ -it : 표준 입력 연결 유지(i) 및 유사 터미널 할당(t)

```
PS D:\DockerClass\GuestBook4ExpressJS> docker container exec guestbookapp pwd
/app
PS D:\DockerClass\GuestBook4ExpressJS> docker container exec -it guestbookapp sh
# cd ..
# ls
app bin boot dev etc home lib lib64 media mnt opt proc root run sbin
# exit
```

30

Docker 컨테이너 관리 주요 명령 6

■ 컨테이너에 파일 복사

- ✓ 컨테이너 끼리 또는 컨테이너와 호스트간 파일 복사
- ✓ 주로 디버깅 중 컨테이너 안에서 생성된 파일을 호스트로 가져와 확인하는 용도
- ✓ docker container cp <컨테이너ID/컨테이너명>:원본 파일 <대상 파일>
- ✓ docker container cp <호스트>_원본 파일 <컨테이너ID/컨테이너명>:대상 파일

■ 리소스 사용 현황 확인

```
docker container cp header.ejs guestbookapp:/app/views/header.ejs
docker container exec -it guestbookapp sh
```

- ✓ 컨테이너 단위로 시스템 리소스 사용 현황 확인
- ✓ Docker container stats <옵션> <대상 _ 컨테이너ID ...>

CONTAINER ID	NAME	CPU %	MEM USAGE / LIMIT	MEM %	NET I/O	BLOCK I/O	PIDS
7c6426ba822e	guestbookapp	0.00%	30.17MiB / 1.952GiB	1.51%	14.4kB / 6.29kB	1.09MB / 8.19kB	23
9ab24226bf94	suspicious_bose	0.00%	32.45MiB / 1.952GiB	1.62%	3.76kB / 1.15kB	20.5kB / 4.1kB	23

31

실습 - 도커 명령으로 컨테이너 다루기

Docker 네트워크

- 격리된 환경의 컨테이너들 간의 통신
- 도커의 기본 네트워크 목록
 - ✓ `docker network ls`
- 네트워크 종류
 - ✓ bridge, host, overlay
- 네트워크 만들기
 - ✓ `docker network create <net-name>`
- 네트워크 정보 확인
 - ✓ `docker network inspect <net-name>`
- 네트워크에 컨테이너 연결
 - ✓ `docker network connect <net-name> <con-name>`
 - ✓ `docker run -itd --name <con-name> --network <net-name> <img-name>`
- 네트워크 연결 해제
 - ✓ `docker network disconnect <net-name> <con-name>`
- 네트워크 제거
 - ✓ `docker network rm <net-name>`

```
tony@vmcontosobuild:~$ sudo docker network ls
NETWORK ID          NAME                DRIVER              SCOPE
3000e0ba80ce        bridge             bridge              local
cfe3e7b6317d        host               host                local
0d4fb992d88f        none              null                local
```

33

도커 명령 줄 도구 사용법

- 1 도커 명령 기본 사용법
`docker <관리명령> <하위명령> <옵션>`
 → `docker container run`
 축약형
 → `docker run`
- 2 관리명령 전체 도움말
`docker help`
- 3 특정 관리 명령 하위명령 전체 도움말
`docker <관리명령> --help`
 → `docker image --help`
- 4 특정 하위명령의 도움말
`docker <관리명령> <하위명령> --help`

```
PS D:\DockerClass> docker image build --help
Usage: docker image build [OPTIONS] PATH | URL | -
Build an image from a Dockerfile

Options:
  --add-host list      Add a custom host-to-IP mapping (host:ip)
  --build-arg list     Set build-time variables
  --cache-from strings Images to consider as cache sources
  --cgroup-parent string Optional parent cgroup for the container
  --compress           Compress the build context using gzip
  --cpu-period int     Limit the CPU CFS (Completely Fair Scheduler) period
  --cpu-quota int      Limit the CPU CFS (Completely Fair Scheduler) quota
  -c, --cpu-shares int CPU shares (relative weight)
  --cpuset-cpus string CPUs in which to allow execution (0-3, 0,1)
  --cpuset-mems string MEMs in which to allow execution (0-3, 0,1)
  --disable-content-trust Skip image verification (default true)
  -f, --file string    Name of the Dockerfile (Default is 'PATH/Dockerfile')
  --force-rm           Always remove intermediate containers
  --iidfile string     Write the image ID to the file
  --isolation string   Container isolation technology
  --label list         Set metadata for an image
  ..                  ..
  ..                  ..
```

34

실습 - 도커 명령으로 도커 네트워크 다루기

도커 컴포즈 다루기



Docker Compose 개요

- 도커는 애플리케이션 배포 전용 컨테이너다.
- 도커 컨테이너 = 단일 애플리케이션
- 실용적인 시스템 구축
 - ✓ 애플리케이션간 연동
 - ✓ 하나 이상의 컨테이너가 서로 통신 하며, 의존 관계 형성
 - 컨테이너 동작 제어를 위한 설정파일이나 환경 변수 전달
 - 컨테이너 의존관계를 고려한 포트 포워딩 설정
- 해결책 → Docker Compose
 - ✓ yml(야물?) 문서에 기술한 설정으로 여러 컨테이너 실행 관리

```
PS D:\DockerClass> docker-compose version
docker-compose version 1.24.1, build 4667896b
docker-py version: 3.7.3
CPython version: 3.6.8
OpenSSL version: OpenSSL 1.0.2q 20 Nov 2018
```

37

docker-compose.yml 구조

- ① Version - yml 해석 문법 버전
- ② Services - 실행 서비스 정의
- ③ 서비스 이름 - ex) db
 - ✓ image - Docker 이미지
 - ✓ ports - 포트 포워딩 설정
 - ✓ links - 참조할 다른 컨테이너
 - ✓ environment - 컨테이너 실행 환경변수
 - ✓ build - Dockerfile 상대 경로
 - ✓ volumes - 호스트와 컨테이너 사이의 파일 공유, 상대경로 가능
 - ✓ depends_on - 서비스가 하나 이상일 때 실행 의존성 지정

```
F: > DockerClass > DockerCompose > docker-compose.yml
1  version: "2"
2  services:
3    wordpress:
4      image: wordpress
5      links:
6        - db:mysql
7      ports:
8        - 8080:80
9    db:
10     image: mariadb
11     environment:
12       MYSQL_ROOT_PASSWORD: Pa55w.rd
```

38

docker-compose 명령

- up - 이미지 가져오거나/빌드, 실행
 - ✓ 서비스에 사용할 네트워크 설정
 - ✓ 필요한 볼륨 생성
 - ✓ 이미지 가져오기(pull)
 - ✓ 이미지 빌드 (--build)
 - ✓ 서비스 의존성에 따라 서비스 실행
 - ✓ -d: 백그라운드 실행
 - ✓ --force-recreate: 컨테이너를 지우고 새로 만들
- Ps - 현재 환경에서 실행 중인 서비스 표시
- stop, start - 서비스 중지 또는 시작
- down - 서비스 삭제 (컨테이너, 네트워크, 볼륨)
- exec - 실행 중인 컨테이너에 명령 실행
- logs - 서비스의 로그 확인.

39

실습 - 도커 컴포즈로 컨테이너 다루기

공감

구독하기

'교육&세미나' 카테고리의 다른 글

웹 프로그래밍 개요 - DAY 1 (0)

2026.06.22

관련글

웹 프로그래밍 개...

프리지아 랩

프리지아(Phrygia)는 소아시아의 고대 지역으로 수도는 고르디온이며, '손에 닿는 것을 모두 황금으로 바꾸'는 미다스왕의 전설로 유명하다. 프리지아 랩은 스마트 라이프를 추구하는 라이프 스타일 INNOVATOR다. 사소하고 일상적인 것을 새로운 가치로 재탄생 시킨다.

구독하기

댓글 0



이름

.....

내용을 입력하세요.

등록

Designed by 티스토리

© AXZ Corp.

